

LINQ DefaultIfEmpty Method in C#

In this article, I will discuss the **LINQ DefaultIfEmpty Method in C#** with Examples using Method Syntax and Query Syntax. Please read our previous article, discussing the [LINQ Single and SingleOrDefault Methods in C#](#) with Examples.

LINQ DefaultIfEmpty Method in C#:

The DefaultIfEmpty method in LINQ is used in a sequence of values to return a default value if the sequence is empty. This method is commonly used with other operators, such as Select and SelectMany, to ensure that a query returns a result even if the source sequence contains no elements.

If the sequence or data source on which the DefaultIfEmpty method is called is not empty, then the original sequence or data source values will be returned. On the other hand, if the sequence or data source is empty, it returns a sequence with the default values based on the data type. There are two overloaded versions available for this DefaultIfEmpty method in LINQ. They are as follows.

```
public static IEnumerable<TSource> DefaultIfEmpty<TSource>(this IEnumerable<TSource> source);  
public static IEnumerable<TSource> DefaultIfEmpty<TSource>(this IEnumerable<TSource> source,  
TSource defaultValue);
```

The first overloaded version does not take any parameter, and in this case, if the sequence is empty, it will return the default values based on the data type. That means this method returns the elements of the specified sequence or the type parameter's default value if the sequence is empty.

You can pass the default value in the second overloaded version of the DefaultIfEmpty method. If the sequence is empty, then this default value (what you pass to the method) will be returned by the DefaultIfEmpty method. That means this method returns the elements of the specified sequence or the specified value if the sequence is empty.

Example to Understand LINQ DefaultIfEmpty Method in C#:

Let us see an example of the LINQ DefaultIfEmpty Method in C# using both Method and Query Syntax. In the following example, the sequence is not empty. So, it is going to return a copy of the original values.

```
using System.Linq;  
using System;  
using System.Collections.Generic;  
namespace LINQDefaultIfEmptyMethodDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //Sequence is not empty  
            List<int> numbers = new List<int>() { 10, 20, 30 };  
        }  
    }  
}
```

```

values        //DefaultIfEmpty Method will return a new sequence with existing :
//Using Method Syntax
IEnumerable<int> resultMS = numbers.DefaultIfEmpty();
//Using Query Syntax
IEnumerable<int> resultQS = (from num in numbers
                             select num).DefaultIfEmpty();
//Accessing the new sequence values using for each loop
foreach (int num in resultMS)
{
    Console.WriteLine($"{num} ");
}
Console.ReadLine();
}
}
}

```

Output: 10 20 30

Example to Understand DefaultIfEmpty Method when Sequence is Empty:

Let us see an Example to Understand the DefaultIfEmpty Method when the Sequence is Empty. In the below example, the sequence is empty. So, in this case, it will return 0 as the default value. This is because 0 is the default value for the integer data type.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQDefaultIfEmptyMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Sequence is empty
            List<int> numbers = new List<int>();
            //DefaultIfEmpty Method will return a new sequence with one element
            //as the Sequence is Empty
            //Using Method Syntax
            IEnumerable<int> resultMS = numbers.DefaultIfEmpty();
            //Using Query Syntax
            IEnumerable<int> resultQS = (from num in numbers
                                         select num).DefaultIfEmpty();
            //Accessing the new sequence values using for each loop
            foreach (int num in resultMS)
            {
                Console.WriteLine($"{num} ");
            }
            Console.ReadLine();
        }
    }
}

```

```

    }
}

```

Output: 0

How to Supply User-Given Values when the Sequence is Empty?

In the following example, the sequence is empty, but we have supplied a default value (i.e., 5) to the `DefaultIfEmpty` method. So, in this case, the default value that we supplied (5) will be returned by the `DefaultIfEmpty` method.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQDefaultIfEmptyMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Sequence is empty
            List<int> numbers = new List<int>();
            //DefaultIfEmpty Method will return 5 as the Sequence is Empty
            //as the Sequence is Empty
            //Using Method Syntax
            IEnumerable<int> resultMS = numbers.DefaultIfEmpty(5);
            //Using Query Syntax
            IEnumerable<int> resultQS = (from num in numbers
                                         select num).DefaultIfEmpty();
            //Accessing the new sequence values using for each loop
            foreach (int num in resultMS)
            {
                Console.Write($"{num} ");
            }
            Console.ReadLine();
        }
    }
}

```

Output: 5

What Happens if the Sequence is not Empty and we have Supplied a Value to the `DefaultIfEmpty` method?

If we supplied a default value, but the sequence is not empty, then, in that case, the original values present in the sequence will be returned. In the below example, we have supplied a default value, i.e., 5, to the `DefaultIfEmpty` method, but the sequence is not empty. So, in this case, the elements present in the sequence will be returned.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQDefaultIfEmptyMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Sequence is not empty
            List<int> numbers = new List<int>() { 10, 20, 30 };
            //DefaultIfEmpty Method will return the Original Sequence values
            //as the Sequence is not Empty
            //Using Method Syntax
            IEnumerable<int> resultMS = numbers.DefaultIfEmpty(5);
            //Using Query Syntax
            IEnumerable<int> resultQS = (from num in numbers
                                         select num).DefaultIfEmpty();
            //Accessing the new sequence values using for each loop
            foreach (int num in resultMS)
            {
                Console.Write($"{num} ");
            }
            Console.ReadLine();
        }
    }
}

```

Output: 10 20 30

LINQ DefaultIfEmpty Method with Complex Type in C#:

Let us see an example of how to use the LINQ DefaultIfEmpty Method with Complex Type in C#. For this, we are going to use the following Employee class. So, first, create a class file with the name **Employee.cs** and then copy and paste the following code into it. It is a simple class with 4 properties and one method to return a collection of employees.

```

using System.Collections.Generic;
namespace LINQDefaultIfEmptyMethodDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public int Salary { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            return new List<Employee>()
            {

```

```

        new Employee { ID = 1, Name = "Preety", Salary = 10000, Gender
"Female"},
        new Employee { ID = 2, Name = "Priyanka", Salary = 20000, Gender
"Female"},
        new Employee { ID = 3, Name = "Anurag", Salary = 35000, Gender
        new Employee { ID = 4, Name = "Pranaya", Salary = 45000, Gender
    };
    }
}
}

```

Next, modify the Main method as follows. Here, we are creating a sequence by calling the GetAllEmployees method of the Employee class, which will return 4 employees. That means our sequence is not empty. Then, we created one employee object and passed that employee object to the DefaultIfEmpty method in case the sequence is empty. In this example, the DefaultIfEmpty method will return the original sequence values as the sequence is not empty.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQDefaultIfEmptyMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Sequence is not empty
            List<Employee> employees = Employee.GetAllEmployees();
            //Create an Employee Object to pass into the DefaultIfEmpty method
sequence is Empty
            Employee emp5 = new Employee() { ID = 5, Name = "Hina", Salary = 1
Gender = "Female" };
            //DefaultIfEmpty Method will return the Original Sequence values
            //as the Sequence is not Empty
            //Using Method Syntax
            IEnumerable<Employee> resultMS = employees.DefaultIfEmpty(emp5);
            //Using Query Syntax
            IEnumerable<Employee> resultQS = (from employee in employees
                                             select employee).DefaultIfEmpty(emp5);
            //Accessing the new sequence values using for each loop
            foreach (Employee emp in resultMS)
            {
                Console.WriteLine($"ID:{emp.ID}, Name:{emp.Name}, Gender:{emp.
Salary:{emp.Salary} ");
            }
            Console.ReadLine();
        }
    }
}

```

You will get the following output when you run the above application code.

```
ID:1, Name:Preety, Gender:Female, Salary:10000
ID:2, Name:Priyanka, Gender:Female, Salary:20000
ID:3, Name:Anurag, Gender:Male, Salary:35000
ID:4, Name:Pranaya, Gender:Male, Salary:45000
```

Now, modify the Main method as follows. Here, you can see we are creating the sequence as Empty. So, in this case, the employee object we passed to the DefaultIfEmpty Method will be returned.

```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQDefaultIfEmptyMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Sequence is empty
            List<Employee> employees = new List<Employee>();
            //Create an Employee Object to pass into the DefaultIfEmpty method
            //sequence is Empty
            Employee emp5 = new Employee() { ID = 5, Name = "Hina", Salary = 10000,
            Gender = "Female" };
            //DefaultIfEmpty Method will return the Employee Object that we passed
            //as the Sequence is Empty
            //Using Method Syntax
            IEnumerable<Employee> resultMS = employees.DefaultIfEmpty(emp5);
            //Using Query Syntax
            IEnumerable<Employee> resultQS = (from employee in employees
                                             select employee).DefaultIfEmpty(emp5);
            //Accessing the new sequence values using for each loop
            foreach (Employee emp in resultMS)
            {
                Console.WriteLine($"ID: {emp.ID}, Name: {emp.Name}, Gender: {emp.Gender},
                Salary: {emp.Salary} ");
            }
            Console.ReadLine();
        }
    }
}
```

Output: **ID: 5, Name: Hina, Gender: Female, Salary: 10000**

LINQ DefaultIfEmpty in Left Outer Join:

DefaultIfEmpty is particularly useful in the context of left outer joins in LINQ. It ensures that, if there are no matching elements in the join, the result will still include the elements from the left sequence, paired with a default value for the right sequence. Here's an example of how it might be used in a left outer join:

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class Program
    {
        static void Main()
        {
            //Using DefaultIfEmpty Method which does not take any parameter
            var query1 = from employee in Employee.GetAllEmployees() //Left Data Source
                        join address in Address.GetAddress() //Right Data Source
                        on employee.AddressId equals address.ID //Inner Join (
                        into EmployeeAddressGroup //Performing LINQ Group Join
                        from address in EmployeeAddressGroup.DefaultIfEmpty()
//Performing Left Outer Join
                        select new //Projecting the Result to Anonymous Type
                        {
                            EmployeeId = employee.ID,
                            Name = employee.Name,
                            Addrees = address?.AddressLine ?? "NA" //Check for
Reference Exception
                        };
            //Using Second Overloaded Version of DefaultIfEmpty Method which takes
default value as a parameter
            var query2 = from employee in Employee.GetAllEmployees() //Left Data Source
                        join address in Address.GetAddress() //Right Data Source
                        on employee.AddressId equals address.ID //Inner Join (
                        into EmployeeAddressGroup //Performing LINQ Group Join
                        from address in EmployeeAddressGroup.DefaultIfEmpty(new
{ AddressLine = "Address Not Available" }) //Performing Left Outer Join
                        select new //Projecting the Result to Anonymous Type
                        {
                            EmployeeId = employee.ID,
                            Name = employee.Name,
                            Addrees = address.AddressLine
                        };
            foreach (var item in query1)
            {
                Console.WriteLine($"EmployeeId: {item.EmployeeId}, Name: {item.Name},
Address: {item.Addrees}");
            }

            Console.ReadKey();
        }
    }

    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
    }
}

```

```

public int AddressId { get; set; }
public static List<Employee> GetAllEmployees()
{
    return new List<Employee>()
    {
        new Employee { ID = 1, Name = "Preety", AddressId = 1},
        new Employee { ID = 2, Name = "Priyanka", AddressId = 2},
        new Employee { ID = 3, Name = "Anurag", AddressId = 0},
        new Employee { ID = 4, Name = "Pranaya", AddressId = 0},
        new Employee { ID = 5, Name = "Hina", AddressId = 5},
        new Employee { ID = 6, Name = "Sambit", AddressId = 6}
    };
}
}
public class Address
{
    public int ID { get; set; }
    public string AddressLine { get; set; }
    public static List<Address> GetAddress()
    {
        return new List<Address>()
        {
            new Address { ID = 1, AddressLine = "AddressLine1"},
            new Address { ID = 2, AddressLine = "AddressLine2"},
            new Address { ID = 5, AddressLine = "AddressLine5"},
            new Address { ID = 6, AddressLine = "AddressLine6"},
        };
    }
}
}

```

Output:

```

EmployeeId: 1, Name: Preety, Address: AddressLine1
EmployeeId: 2, Name: Priyanka, Address: AddressLine2
EmployeeId: 3, Name: Anurag, Address: NA
EmployeeId: 4, Name: Pranaya, Address: NA
EmployeeId: 5, Name: Hina, Address: AddressLine5
EmployeeId: 6, Name: Sambit, Address: AddressLine6

```

In this join, if an Employee has no matching Address in the Addresses collection, `DefaultIfEmpty` will insert a default value (null in this case), and the `??` operator is used to provide a fallback value ("NA") when rendering the result. In the second overloaded version, we have specified a default when the Address is not available.

When to Use the LINQ `DefaultIfEmpty` Method in C#?

You should use the LINQ `DefaultIfEmpty` method in C# to ensure that you always have at least one element in the result of a LINQ query, even if the source collection might be empty. This method is particularly useful

in the following scenarios:

- **Left Outer Joins:** LINQ has no explicit left outer join operation, but you can simulate it using a group join (GroupJoin) followed by SelectMany and DefaultIfEmpty. This ensures that for every element in the outer sequence if there are no matching elements in the inner sequence, you still get a result with the outer element and a default value for the inner element.
- **Providing Default Values:** When you have an empty sequence, you need to ensure that the subsequent query operators have a value to work with. DefaultIfEmpty allows you to specify a default value that can be more meaningful in context than a simple null or zero.
- **Coalescing Empty Results:** In data processing, when an empty sequence would cause issues down the line, using DefaultIfEmpty can coalesce the empty result to a single default item, allowing for smoother processing and avoiding the need for additional null-checking.
- **Fallback Scenarios:** In cases where you expect a sequence always to have data, but due to some exceptional condition, it doesn't, DefaultIfEmpty can be used to provide a fallback value, which can indicate the absence of expected data.
- **Ensuring Collection Materialization:** When using certain LINQ providers (like Entity Framework), an empty collection might not materialize as an instance but remain null. Using DefaultIfEmpty can ensure that you always get an IEnumerable instance, even if it contains only a default value, which can prevent NullReferenceException.
- **Single Element Processing:** If you want to ensure that a single-element operation (like Single or First) does not throw an exception on an empty sequence, preface it with DefaultIfEmpty. It guarantees at least one element to process, which will be the default value if the sequence is otherwise empty.

It's important to note that DefaultIfEmpty will produce a sequence with a single element, a default value of the type of elements in the sequence if the original sequence is empty. If the sequence is not empty, DefaultIfEmpty has no effect, and the original sequence is returned unchanged.

In the next article, I will discuss the [LINQ SequenceEqual Operator in C#](#) with Examples. In this article, I try to explain the **LINQ DefaultIfEmpty Method in C#** with Examples. I hope you understand the need and use of the LINQ DefaultIfEmpty Method in C# with Examples.